36、软、硬件调试方法总结

全志V3S嵌入式驱动开发（软、硬件调试方法总结）

原创

嵌入式-老费 已于 2023-08-29 15:26:28 修改

阅读量1.7k 收藏 5

点赞数 1

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

基于全志soc的linux嵌入式开发专栏收录该内容

73 篇文章¥49.90¥99.00

已订阅8折续费

[

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 ›

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

摘要 本文介绍了全志V3S soc的驱动开发和调试经验，包括电源、晶振的重要性，基于SD卡和U盘的Linux开发调试方法，以及驱动放置策略，强调了对比法在解决驱动问题中的应用。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

v3s开发板本身是比较小巧的，ddr也是嵌入在soc里面的，电路设计也不复杂，因此很受电子爱好者的喜欢。不过大部分朋友一般都是自己把电路板设计出来，然后烧入别人的镜像测试的。这样虽然也有一定的可玩行，不过还是建议大家可以自己多多熟悉嵌入式软件方面的内容，这样或许更有乐趣的多。下面结合近一段的调试经历，总结一下linux+soc调试的经验。

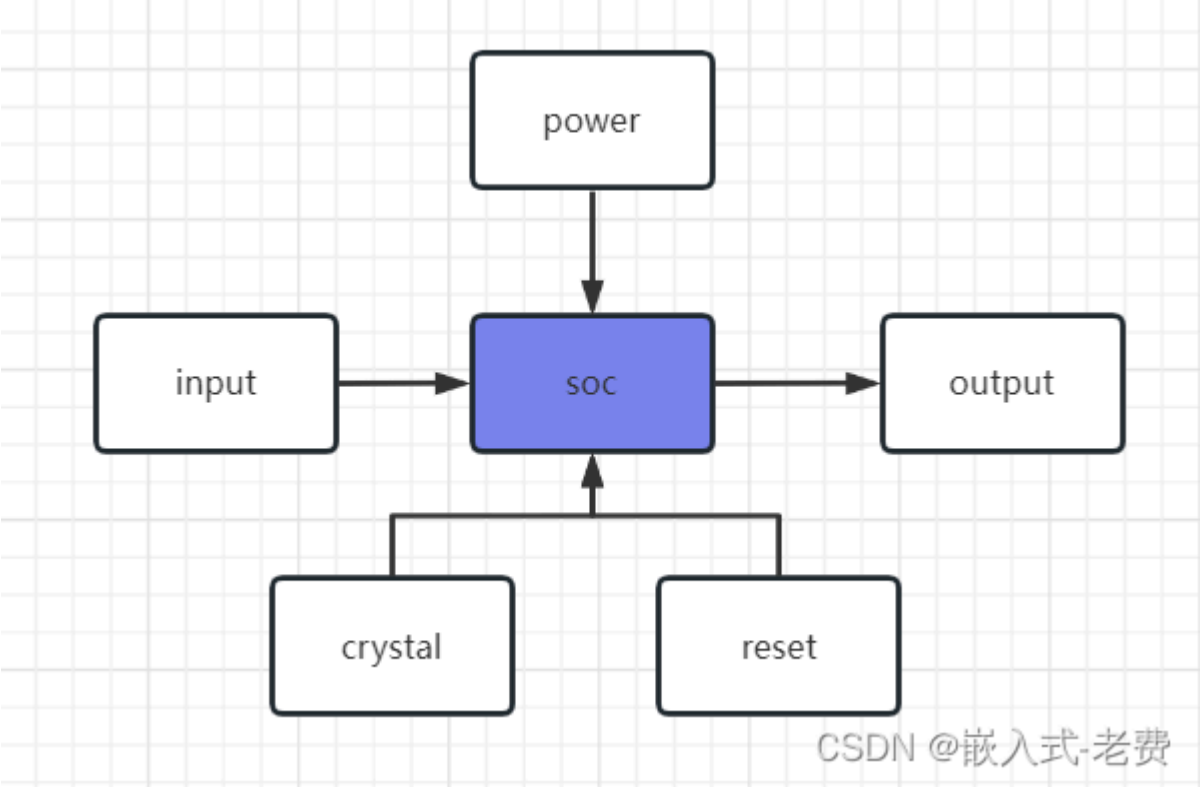
1、soc电源十分重要

soc上面电源很多，比如一颗soc，就可能有3.3v、2.5v、1.8v、1.2v等多种规格的电压。好一点的soc，厂家会搭配一个pmu。如果没有pmu，也会用ea3036这样的芯片，做一个简单的电源管理电路。实在没有办法了，才会选用不同的dcdc，调制不同的电压，这些都是可以的。

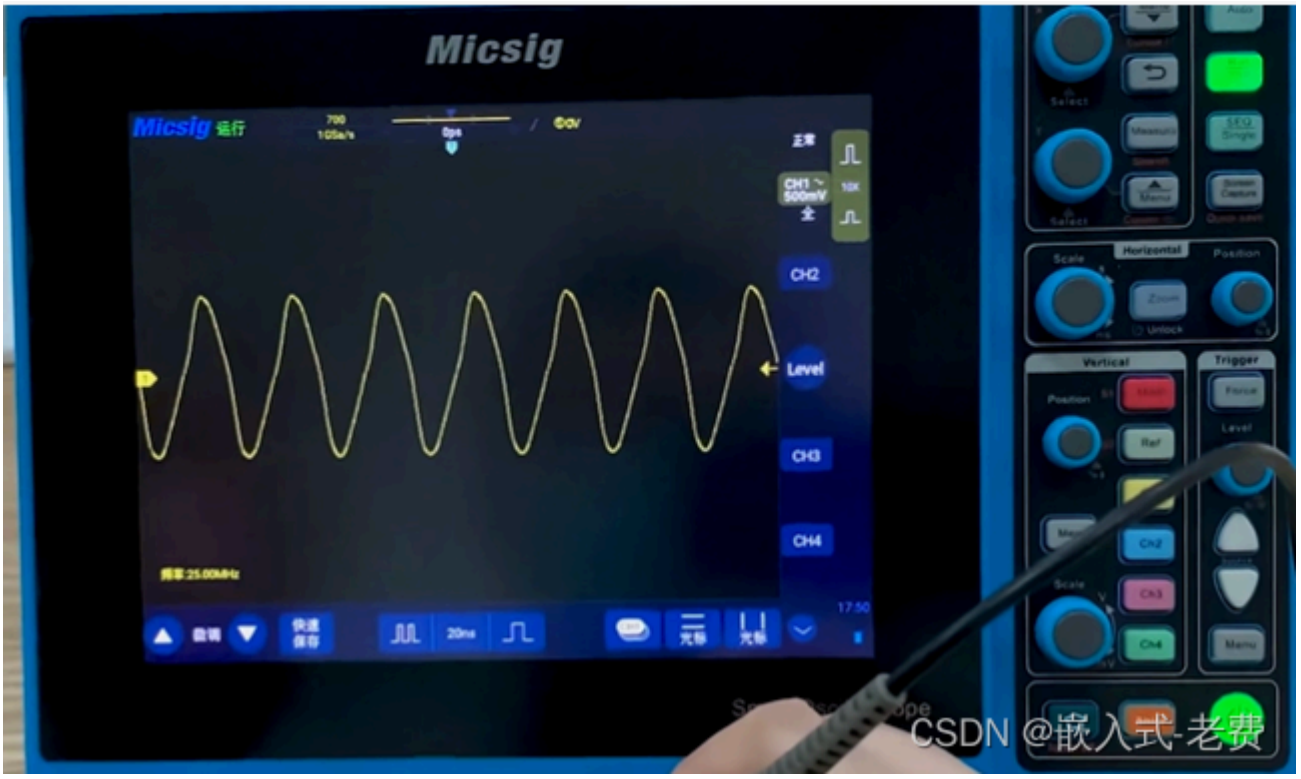
电源是电路的基础，没有电源一切都无从谈起。如果板子不工作了，我们的第一个反应往往也是用万用表查一下电压，看看哪里短路了，哪里断路了，这些都是常用测试的方法。毕竟没有电源，信号的有无、信号的质量，这些都无从谈起。

2、晶振

v3s板子上面有三个晶振。两个是soc的，一个是wifi电路的。其中soc的两个晶振用途也各不相同，一个是作为倍频pll使用，一个是作为rtc使用。相比较而言，后者的精度更高一点。而wifi模块一般也会要求搭一个晶振模块。除了电源之外，晶振也是比较容易出问题的地方。倒不是晶振电路有多难，主要是在电商网站上面很容易买到假的晶振，这个也是经常发生的现象。解决的办法就是去正规旗舰店购买，贵一点，但好歹是真的。



有过嵌入式开发经验的朋友都知道，电源、晶振、复位、输入、输出，这是系统boot起来的5个基本条件。电源前面已经谈过，复位比较简单，输入输出可以用按键和点灯代替，所以这里如果搞定了晶振，用示波器测出了晶振的信号，那么整个最小电路基本就ok了。



3、基于sd卡开发linux

目前主要的soc基本都支持micro sd卡启动。 当然，除了micro sd卡，他们也支持spi-nor、spi-nand、usb、jtag等启动方法。整体来说，micro sd卡调试是最方便的。

在最初一行代码没有的时候，就可以把uboot编译生成的bin文件用ubuntu的dd命令写入到micro sd卡上，这是非常便捷的方法。

此外，micro sd卡还有很多其他的优势。比如，micro sd卡很大，一般来说，它要比spi-nor、spi-nand、emmc大得多，最小的都有8g、16g。因此哪怕最终发布嵌入式系统的时候，我们不用micro sd卡启动，但是在开发的时候，最好还是用micro sd卡，毕竟太方便了。它也不需要jtag工具，有一个虚拟机，有一个usb的读卡器，这样就可以进行linux开发了，从uboot到kernel、rootfs、app，就没有它不能干的。

等micro sd卡port好linux、rootfs之后，就可以慢慢引入spi-nor、spi-nand驱动，直到生成支持spi-nor、spi-nand的uboot和image，这个时候再从flash启动uboot和linux系统，开发过程就会容易很多。

不同的boot方式，就是uboot是不一样的，linux和rootfs基本差不多的。 所以sd卡boot方式下，如果搞定了大部分驱动和应用软件开发，那么产品只要简单切换到spi-nor、spi-nand、emmc启动就可以了。

4、不要把所有的驱动都放在zImage

所有的驱动都放到zImage，一个最大的坏处就是，这个zImage会越来越大。这样就要求我们在后期移植spi-nor、spi-nand的时候，需要不停调整image的布局。dtb留多大，zImage留多大，rootfs留多大等等，一个大小变了，其他就要跟着改变，很麻烦，关键是还比较容易出错。所以，关于内核功能和驱动这一部分，可以把不常用的驱动、或者比较大的驱动放在文件系统里面，或者挂载到外面存储设备，等rootfs后面起来之后，动态加载，这也是可以的。

5、基于u盘的用户程序调试

说完了zImage，下面就是用户程序。不是所有嵌入式最终都可以接受micro sd卡启动的，毕竟如果是运动的嵌入式设备、或者工业设备，这个micro sd卡还是存在一定的风险的。但是spi-nor和spi-nand又比较小，空间也不大，这就是一个问题，调试会很不方便。比如，spi-nor一般也就32MB，而spi-nand则为128M，可大一点的文件就会占据很大的空间。

所以这个时候，一个比较好的方法，就是把程序、动态库和配置文件copy到u盘上。等嵌入式起来之后，把u盘mount到/mnt目录，这样就可以继续后面的测试了。

```
ount /dev/sda1 /mnt
cd /mnt/v3s
```

中间依赖的动态库如果也在u盘上面，那么LD_LIBRARY_PATH也要做修改，

```
export LD_LIBRARY_PATH=/mnt/v3s/tslib/lib:$LD_LIBRARY_PATH
```

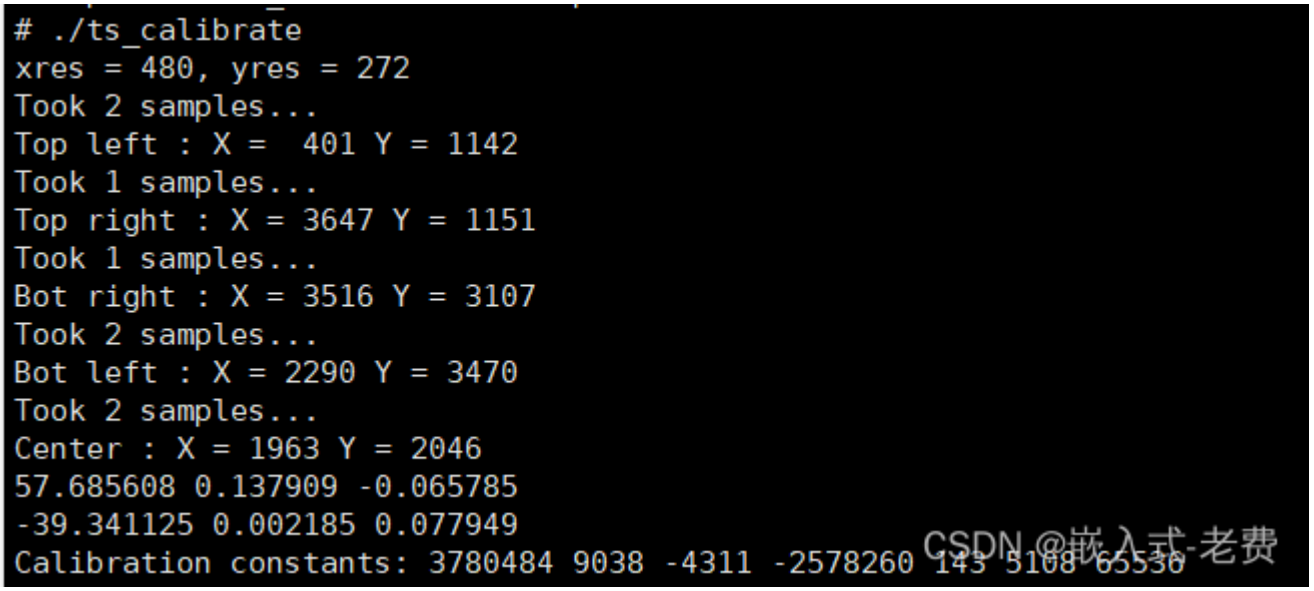
windows的很多文件系统，比如fat可能不支持软链接这种形式。其实也没有关系，直接cp文件，换一个文件名就可以了，比如像这样，

```
cp libts.so.0.10.4 libts.so.0
```

做好了这些准备，u盘上有一些配置文件的话，那么涉及到的环境变量也需要修改一下，

```
export TSLIB_CONFFILE=/mnt/v3s/tslib/etc/ts.conf
export TSLIB_PLUGINDIR=/mnt/v3s/tslib/lib/ts
```

做到了上面这些内容，基本上u盘上面的程序也可以正常运行了，使用还是很方便的。下图就是利用u盘上的ts_calibrate进行标定的过程打印，



6、勤用对比法看驱动问题

实际开发的时候，我们经常遇到这样一个问题。那就是相同的驱动，升级了内核之后，之前好的功能就不行了，这个时候就要勤用对比法，看看之前的配置是什么，现在的配置是什么，两者有什么差别。比如之前wifi驱动的问题，关于配置这块，如果用对比法，其实很容易看出区别的。因为kernel 5.2.y是这么配置的，

```
--- Wireless
<M>   cfg80211 - wireless configuration API
[ ]     nl80211 testmode command
[ ]     enable developer warnings
[*]     enable powersave by default
[ ]     cfg80211 DebugFS entries
[ ]     cfg80211 wireless extensions compatibility
<M>   Generic IEEE 802.11 Networking Stack (mac80211)
      Default rate control algorithm (Minstrel)  --->
[ ]   Enable mac80211 mesh networking support
[ ]   Enable LED triggers
[ ]   Export mac80211 internals in DebugFS
[ ]   Trace all mac80211 debug messages
[ ]   Select mac80211 debugging features  ----
```

CSDN @嵌入式-老费

而之前kernel 4.14.y却是这么配置的，

```
--- Wireless
<*>   cfg80211 - wireless configuration API
[ ]     nl80211 testmode command
[ ]     enable developer warnings
[*]     enable powersave by default
[ ]     cfg80211 DebugFS entries
[*]     cfg80211 wireless extensions compatibility
<*>   Generic IEEE 802.11 Networking Stack (mac80211)
      Default rate control algorithm (Minstrel)  --->
[ ]   Enable mac80211 mesh networking (pre-802.11s) support
[*]   Enable LED triggers
[ ]   Export mac80211 internals in DebugFS
[ ]   Trace all mac80211 debug messages
[ ]   Select mac80211 debugging features  ----
```

CSDN @嵌入式-老费

对比下，这样就比较方便比较看出来，真正有区别的地方，就是中间红框有没有选中。